

Introduction to Data Structures

1. A hospital system must store patient records where quick search and updates are required. Which data structure would best suit this requirement and why?
 2. A social media app needs to manage millions of posts efficiently. Explain which data structure strategy would optimize storage and retrieval.
 3. A compiler needs to process nested expressions. Which data structure is most appropriate and how does it help?
-

Linked Lists (Types & Operations)

4. A music playlist allows users to move forward and backward between songs. Which type of linked list would you use and why?
 5. In a memory-constrained system, elements must be dynamically allocated and deallocated. Why would a linked list be preferred over arrays?
 6. A train has coaches that can be attached/detached from both ends. Which linked list type is most suitable? Justify.
 7. You are given a singly linked list where every node points randomly to another node. How would you clone this structure efficiently?
 8. A system frequently inserts elements in the middle of a sequence. Compare linked list vs array performance for this scenario.
-

Stacks (Operations & Applications)

9. A browser implements an “undo” feature for visited pages. How does a stack help implement this functionality?
10. A compiler checks for balanced parentheses in code. Explain how a stack is used in this scenario.

11. In a postfix expression evaluation system, how does a stack process operands and operators?
 12. A system requires reversing a string efficiently. Explain how a stack can be used.
 13. A recursive function is converted to iterative form. Which data structure replaces recursion internally and why?
-

Queues (Operations & Applications)

14. A printer processes print jobs in the order they arrive. Which data structure is used and why?
 15. In a ticket booking system, VIP customers are given priority. Which type of queue would handle this?
 16. A CPU scheduling algorithm uses round-robin scheduling. Explain how a circular queue is used.
 17. In a call center, incoming calls are handled in order but urgent calls jump ahead. Design a queue system for this.
-

Trees (Binary Trees & Balanced Trees)

18. A company hierarchy needs to be represented where each employee reports to one manager. Which tree structure is appropriate?
19. A search system needs fast lookup, insertion, and deletion. Why would a balanced binary search tree be preferred?
20. Given a binary tree, how would you check if it is height-balanced?
21. A database index uses AVL trees. Explain how rotations maintain balance during insertions.
22. In an expression tree, how are infix expressions evaluated using tree traversal?

Graphs (Representation & Algorithms)

23. A navigation system finds the shortest path between two cities. Which graph algorithm would you use and why?
 24. A social network wants to detect friend circles. Which graph traversal technique is suitable?
 25. A task scheduling system must ensure no circular dependencies exist. How would you detect cycles in a graph?
 26. A network routing system uses weighted edges. Explain how Dijkstra's algorithm helps.
 27. Compare adjacency matrix and adjacency list for a sparse graph in a real-time system.
-

Hashing (Applications & Functions)

28. A password system must verify users quickly without storing plain text passwords. How does hashing help?
 29. A system experiences many collisions in a hash table. What strategies can be applied to resolve them?
 30. A large dataset needs constant-time search operations. Design a hashing approach and discuss potential issues.
-

Heaps (Applications & Operations)

31. A hospital emergency system must always treat the most critical patient first. Which heap structure is suitable?
32. In a sorting system, explain how heap sort works using a max heap.

33. A real-time system needs to continuously find the smallest element. Why is a min-heap efficient?
34. A streaming application needs to find the median of incoming numbers. How can heaps be used?